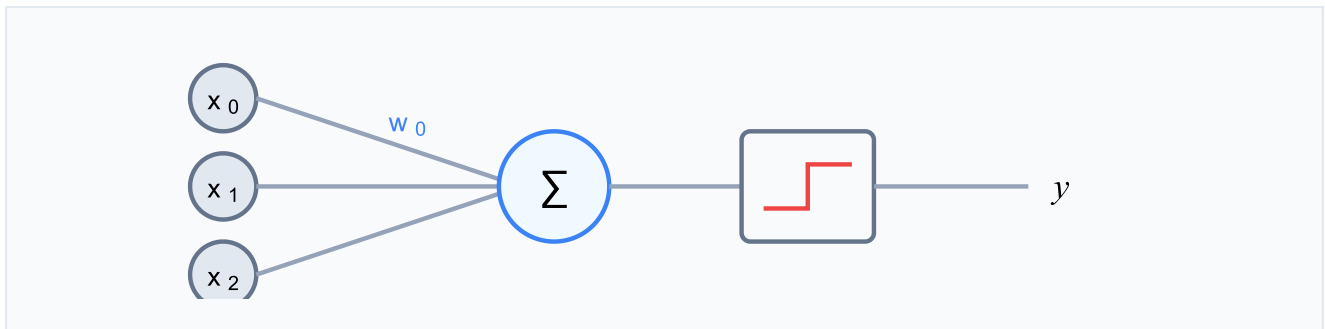


1. The Thresholded Perceptron

The human brain processes information using approximately 10^{11} neurons, with 10^4 connections per neuron. It relies on distributed representation and parallel processing rather than explicit algorithmic coding. The Thresholded Perceptron is the earliest and simplest mathematical model of a single biological neuron. It takes a series of inputs, multiplies each by a specific weight, sums them together, and applies a hard "step" function to decide whether to fire (output 1) or remain dormant (output -1).



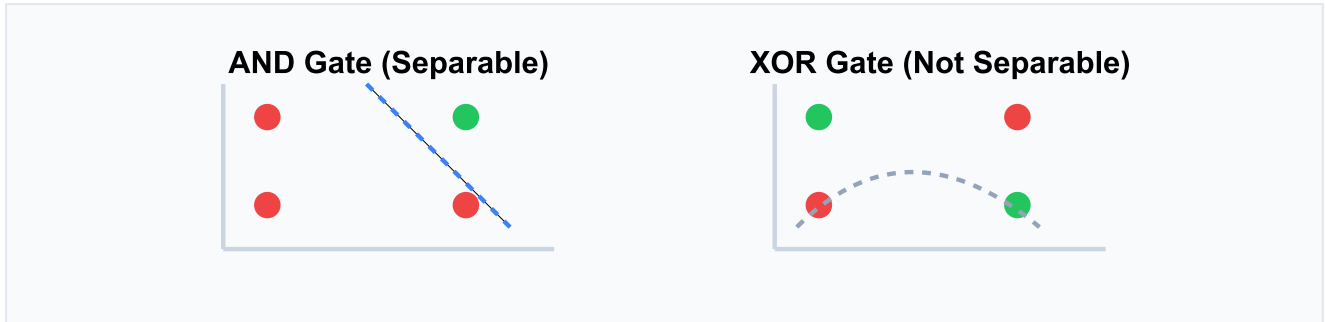
Mathematically, we define the weighted sum as $net = \sum w_i x_i$. The output y is dictated by a step function: $y = 1$ if $net > 0$, and -1 otherwise. During training, if the perceptron makes a mistake, the weights are updated using the Delta rule: $\Delta w_i = \eta(t - o)x_i$, where η is the learning rate, t is the target, and o is the output.

Intuition: The Decision Boundary

Imagine plotting data points on a 2D graph. The perceptron is trying to draw a single, perfectly straight line across the graph. If it outputs 1, the point is on one side of the line; if -1, it is on the other. This straight line is called a *hyperplane*.

The Limits of Linear Separability

Because the thresholded perceptron relies on a straight-line decision boundary, it can only solve problems where the data is **linearly separable**. For example, a logical AND gate is linearly separable. However, it fails completely on an XOR gate, where no single straight line can successfully separate the positive from the negative classes.



Here is how the training loop for a thresholded perceptron is implemented in Python. Notice how the hard step function (outputting exactly 1 or -1) is applied during the forward pass:

```
def fit(self, X, y): # Insert bias term (x0 = 1) X_bias = np.c_[np.ones((X.shape[0], 1)), X] self.w = np.zeros(X_bias.shape[1]) for _ in range(self.epochs): for xi, target in zip(X_bias, y): net = np.dot(xi, self.w) # The Hard Threshold output = 1 if net > 0 else -1 # Perceptron rule delta_w = self.eta * (target - output) * xi self.w += delta_w
```

If the data isn't linearly separable, the weights will oscillate infinitely. Furthermore, if we want to build a deep network by stacking these neurons, the hard threshold function destroys our ability to train them, because its mathematical derivative is exactly zero almost everywhere.

2. Unthresholded Perceptron & Delta Rule

To train multi-layer networks, we must use Gradient Descent. This requires a smooth, continuous error landscape. We achieve this by simply removing the step function. The unthresholded perceptron directly outputs the raw dot product: $o = \text{vec}\{w\} \cdot \text{vec}\{x\}$. Because the output is continuous, we can define a differentiable Cost Function, specifically the Mean Squared Error: $E = \frac{1}{2} \sum (t - o)^2$.



Intuition: Rolling Down a Hill

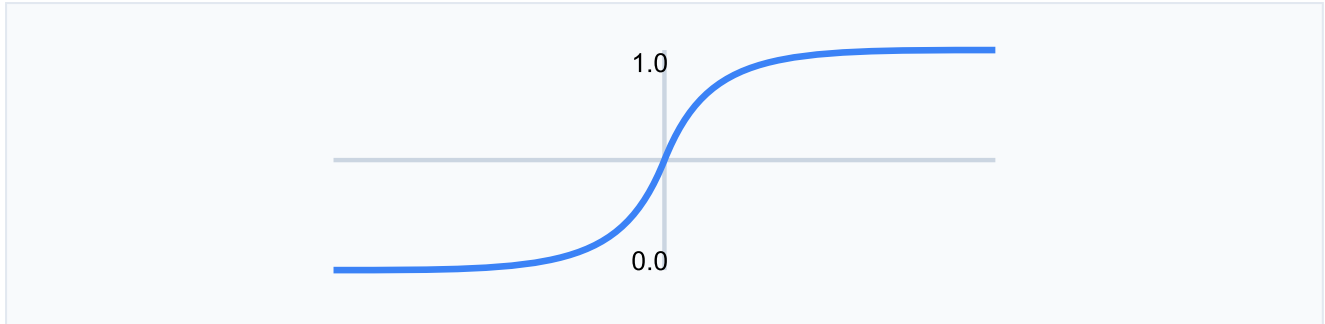
Instead of being told "you are entirely wrong," the network is now told "you are off by 0.4." The squared error function maps out a literal mathematical bowl (a convex surface). By taking the derivative of this error with respect to the weights (the gradient), the network calculates the steepest slope and takes a step downhill.

In code, we calculate the continuous output, measure the exact distance from the target, and compute the gradient over the entire dataset (Batch Gradient Descent) to update our weights smoothly.

```
# Batch Gradient Descent on Squared-Error Cost
outputs = np.dot(X_bias, self.w)
errors = y - outputs # Cost function E: 1/2 * sum(t - o)^2
cost = 0.5 * np.sum(errors ** 2)
# Gradient of E with respect to weights
gradient = -np.dot(X_bias.T, errors) / N
# Update weights using the learning rate
self.w -= self.eta * gradient
```

3. The Sigmoid Unit and Non-Linearity

While the unthresholded linear perceptron allows for gradient descent, it introduces a fatal flaw: stacking multiple linear layers collapses into a single layer mathematically. If Layer 1 applies weight matrix W_1 , and Layer 2 applies W_2 , the output is $y = W_2(W_1x)$. Because matrix multiplication is associative, this is simply $y = W_3x$. The network remains fundamentally linear.



To prevent collapse, we must inject non-linearity between layers. We need a function that mimics the thresholding step (squashing outputs between 0 and 1) but remains perfectly smooth and differentiable for Gradient Descent. The **Sigmoid function** perfectly bridges this gap: $\sigma(x) = 1 / (1 + e^{-x})$.

```
def sigmoid(x): return 1 / (1 + np.exp(-x)) def sigmoid_derivative(output): # The
elegant derivative of sigmoid is simply y(1 - y) return output * (1 - output) # Inside the
training loop: net = np.dot(X_bias, self.w) output = sigmoid(net) # Gradient calculation
includes the derivative delta = error * sigmoid_derivative(output)
```

A beautiful property of the sigmoid function is its derivative, which is extremely cheap to compute mathematically: $y(1 - y)$, where y is the output of the node itself. By utilizing the Sigmoid unit, we successfully combine non-linear problem-solving (capable of learning XOR) with smooth, gradient-based optimization.